BrainAI

ProSuite Industrial

ZERO-TRUST COMMUNICATION

PRODUCT DOCUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 PUBLIC CLASSIFICATION
Core Architecture & Development	**ProChat System Manual**	Access Level: Public / Beta

---

Revision History

Version	Date	Author	Status	Description
1.0.0	2026-01-01	S. Köhne	Stable	Forward the Secret
2.0.0	2026-02-26	S. Köhne	Stable	High-Speed & Zero-Trust Edition (AEAD Header Protection, Noise-Padding, Strict Length Validation)

1. Product Overview

The ProChat Core Engine v2.0.0 is a highly secure communication module developed for use in extreme high-security (Zero-Trust) environments. It ensures the confidentiality, integrity, and authenticity of data streams and protects against modern attack vectors such as replay attacks, traffic analysis, and man-in-the-middle attacks.

Core Features

- **Cryptographic Isolation (Ratchet):** Each session is isolated by its own asynchronously rotating ratchet state.
- **True AEAD & Header Protection:** Both the payload and critical metadata (sequence number, type, exact length) are encrypted and authenticated within a continuous cipher block.
- **ProKey Noise-Padding:** Empty packet space is not filled with zeros, but with true hardware entropy (noise) to make traffic analysis and length inference physically impossible for attackers.
- **RAM-Vault Technology:** Internal protection of active session keys against memory dumping at runtime.

2. Technical Specifications

2.1 Global Constants

These values are mandatory for all implementations and guarantee constant memory complexity:

- **PC_PKT_SIZE:** *1024* Bytes (Fixed transmission size for absolute obfuscation of metadata and payload sizes).
- **PC_MAX_PAYLOAD:** *988* Bytes (Maximum available payload per packet).
- **Key/Seed Size:** Exactly *32* Bytes (*256*-bit entropy).

2.2 Packet Layout (Public Interface)

Externally, a ProChat packet is a completely opaque binary block of 1024 bytes. Internally, it is structured as follows:

- **Routing-Tag:** *4* Bytes (Plaintext tag for O(1) session identification by the receiver).
- **Encrypted Header & Payload:** *1004* Bytes (Fully encrypted area starting at byte 4. Contains sender ID, TX counter, message type, the exact 16-bit payload length, as well as the actual payload and dynamic entropy padding).
- **Auth-Tag (MAC):** *16* Bytes (Integrity assurance of the entire packet via ProHash Constant-Time HMAC).

3. Global Error Management

The SDK uses a hybrid error model. While the native engine returns numerical error codes, the wrappers transform these into language-specific paradigms (exceptions or status objects). Version 2.0.0 integrates hard boundary checks directly at the core level.

Code	***Constant***	***Description:***	***Criticality***
0	PC_OK	Operation completed successfully.	*None*
-1	PC_ERR_PARAM	Invalid buffer lengths, oversized payloads, or buffer overflow prevention.	*Medium (Logic Error)*
-2	PC_ERR_AUTH	Constant-time MAC comparison failed. Packet manipulated!	*High (Attack!)*
-3	PC_ERR_REPLAY	Packet counter invalid or already processed.	*Medium (Network Error)*
-4	PC_ERR_DESYNC	Ratchet synchronization lost.	*Fatal (Reset required)*

4. C# (.NET) Integration & Exception Handling

The *C# Wrapper* uses an object-oriented model with a central Client class. It abstracts the complex padding and memory operations of the v2.0.0 engine.

4.1 Implementation Example

```
using ProChat;

public void SecureCommunicationExample()
{
    try
    {
        // 1. Initialization with 32-Byte Seed
        byte[] ramSeed = GetSecureEntropy(32);
        var client = new ProChatClient(uid: 1001, seed: ramSeed);

        // 2. Add Peer
        byte[] sharedKey = GetSharedKeyFromHandshake();
        if (!client.AddPeer(2002, sharedKey))
        {
            throw new InvalidOperationException("Peer initialization
        }

        // 3. Send (Engine automatically handles Noise-Padding to 102
        byte[] payload = System.Text.Encoding.UTF8.GetBytes("BrainAI
        byte[] encryptedPacket = client.EncryptMessage(2002, type: 1,

        if (encryptedPacket == null)
        {
            LogError("Encryption error in Core Engine (e.g., Payload
        }

        // 4. Receive and Validate (incl. AEAD-Header Check)
        uint senderId;
        byte[] decrypted = client.DecryptPacket(receivedBytes, out se

        if (decrypted == null)
        {
            // Covers strict manipulations and buffer underflows
            throw new SecurityException("Integrity check failed! Pack
```

```
        }
    }
    catch (ArgumentException ex)
    {
        LogError($"Parameter error: {ex.Message}");
    }
}
```

5. Python (High-Performance Scripting)

The *Python Wrapper* is designed for maximum flexibility. It converts C error codes into clean Python return values.

5.1 Implementation Example

```python
from prochat import ProChat, ProChatStatus

def run_chat_service():
    # Setup Engine
    seed = b'\x12' * 32
    client = ProChat(uid=1001, seed=seed)

    # Peer Setup
    key = b'\xAB' * 32
    if not client.add_peer(peer_uid=2002, key=key):
        print("Error: Could not register peer.")
        return

    # Send Loop (Remaining packet space is filled with cryptographic
    msg = b"System-Health: OK"
    packet = client.encrypt(target_uid=2002, msg_type=1, message=msg)

    if packet:
        send_to_socket(packet)

    # Receive Logic with validated payload length
    raw_data = socket.receive(1024)
    result = client.decrypt(raw_data)

    if result is None:
        # Catches PC_ERR_AUTH (-2) Constant-Time Check failures
        handle_security_incident("Manipulation or Desync detected!")
    else:
        sender_id, payload = result
        print(f"Secure message from {sender_id}: {payload.decode()}")
```

6. Java (JNA / Enterprise Integration)

The *Java* implementation utilizes a static IProChat library instance and offers memory-safe data handling for backend architectures through the `DecryptedMessage` class.

6.1 Implementation Example

```
import com.brainai.prochat.ProChat;

public class SecureGateway {
    public void processIncoming(byte[] packet) {
        ProChat.Client client = new ProChat.Client(1001, mySeed);

        // Decryption attempt (Reads exact payload length from the he
        ProChat.DecryptedMessage msg = client.decryptPacket(packet);

        if (msg == null) {
            System.err.println("Critical Error: Authentication failed
            auditLog.reportAttack(System.currentTimeMillis(), "MAC_FA
            return;
        }

        System.out.println("Packet authenticated from UID: " + msg.se
        processPayload(msg.data);
    }
}
```

7. JavaScript (Node.js / Web-Services)

The *Node.js* wrapper uses native buffers for zero-copy operations and is ideal for scalable backend systems or WebAssembly edge nodes.

7.1 Implementation Example

```
const ProChat = require('./ProChat');

const chat = new ProChat(1001, seedBuffer);

// Send
const packet = chat.encrypt(2002, 1, "Cloud-Status: Green");
if (!packet) {
    console.error("Encryption error - check engine parameters (Payloa
}

// Receive
const result = chat.decrypt(incomingBuffer);
if (!result) {
    // Covers PC_ERR_AUTH (-2)
    console.error("Security Alert: Packet integrity violated!");
} else {
    const { senderUid, payload } = result;
    console.log(`Verified data from ${senderUid}:`, payload.toString(
}
```

8. VB.NET (Industrial Systems)

Specifically for industrial automation systems (OT), the VB.NET wrapper offers full compatibility and strict error handling via `IsNot Nothing` checks to prevent system downtimes caused by buffer overflows.

8.1 Implementation Example

```
Public Sub HandleIndustrialMessage(packet As Byte())
    Dim client As New ProChatClient(1001, globalSeed)
    Dim senderUid As UInteger = 0

    ' Decrypt and Validate (Prevents erroneous length evaluation)
    Dim payload = client.DecryptPacket(packet, senderUid)

    If payload Is Nothing Then
        ' Error handling for PC_ERR_AUTH or PC_ERR_REPLAY
        RaiseSecurityEvent("Integrity error from sender " & senderUid
        Exit Sub
    End If

    ' Processing on success
    LogMessage("Secure data packet received from " & senderUid & ".")
    UpdateMachineState(payload)
End Sub
```

9. Best Practices & Security Guidelines

- **Strict Boundary Control:** The ProChat Engine v2.0.0 rigorously rejects payloads > 988 bytes at the native level to prevent buffer overflows. Applications must respect these limits in advance (e.g., through chunking).
- **Session Synchronicity & KDF:** ProChat uses a highly strict KDF ratchet (Key Derivation Function). For packet-based UDP transmissions in noisy environments, the wrapper *must* implement an external sequence check or buffering to avoid a permanent `PC_ERR_DESYNC` caused by out-of-order packets.
- **Entropy Source:** The engine relies on `ProKey` for noise padding. Ensure that the executing system has sufficient hardware entropy.
- **Memory Clearing:** In languages with garbage collection (Java, C#, JS), sensitive buffers should be explicitly overwritten with zeros after use before being released to the GC.

© 2026 Sascha Köhne	✉️ **Contact:** koehne83@googlemail.com	🛡️ **ProTU-Verified**
CEO / System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**